

PROJECT DOCUMENTATION

GymReserve AI

Cross-Platform Gym Management & AI Workout Recommendation App

PACKAGE

balneg

VERSION

1.0.0

FRAMEWORK

Expo 56

BACKEND

Supabase

Client — Book & Track

Admin — Manage Facility

Trainer — Coach Clients

Table of Contents

Navigate through all sections of this documentation

1	Project Overview	3
2	Tech Stack	3
3	Project Structure	5
4	Architecture & Key Concepts	6
5	Database Schema (Supabase)	7
6	Environment Variables	8
7	Prerequisites	8
8	Running the App — Step by Step	9
9	Building for Production (EAS)	11
10	Supabase Edge Functions	11
11	User Roles & Navigation Flow	12
12	Component Library	12

GymReserve AI (`ba1neg`) is a mobile-first application for managing a gym facility. It supports three user roles with different levels of access and functionality.



Client

- ✓ Browse & reserve equipment
- ✓ Track workout sessions
- ✓ View AI recommendations
- ✓ Manage body metrics & profile
- ✓ Receive push notifications



Admin

- ✓ CRUD gym equipment
- ✓ Manage all reservations
- ✓ View registered users
- ✓ Analytics dashboard
- ✓ Log maintenance records



Trainer

- ✓ Same access as Admin
- ✓ Monitor client activity
- ✓ Manage equipment status
- ✓ Approve/cancel bookings

The app uses **Supabase** as its backend (auth, PostgreSQL database, realtime subscriptions, storage) and **Expo** as the cross-platform framework targeting **Android**, **iOS**, and **Web**.

2 Tech Stack

Core Framework

Technology	Version	Purpose
React Native	0.85.3	Cross-platform mobile UI framework
React	19.2.3	UI component library
Expo	~56.0.12	Managed workflow, build tooling, native APIs
TypeScript	~6.0.3	Static typing across the entire codebase

Routing & Navigation

Package	Purpose
expo-router ^56.2.11	File-based routing — folder structure defines URL paths
react-native-screens	Native screen containers for better performance
react-native-safe-area-context	Safe area insets for notched/dynamic-island devices

Backend & Authentication

Package	Purpose
@supabase/supabase-js ^2.108.2	Supabase client — auth, DB queries, realtime subscriptions
expo-secure-store	Encrypted JWT token storage on native (iOS Keychain / Android Keystore)
expo-auth-session	OAuth session helpers
expo-web-browser	In-app browser for OAuth / deep-link flows
react-native-url-polyfill	URL API polyfill required by Supabase on React Native

UI & Animations

Package	Purpose
expo-linear-gradient	Gradient backgrounds, card overlays

Package	Purpose
<code>react-native-reanimated 4.3.1</code>	Smooth 60fps layout & gesture animations
<code>react-native-svg 15.15.4</code>	SVG rendering used by charts and the ActivityGauge
<code>react-native-chart-kit ^7.0.1</code>	Line, bar, and pie charts for dashboards
<code>lucide-react-native ^1.22.0</code>	Icon library (consistent, clean icon set)

Media, Device & Notifications

Package	Purpose
<code>expo-image-picker</code>	Camera roll & camera access for profile pictures
<code>expo-notifications</code>	Push notification sending & receiving
<code>expo-device</code>	Device information (used for push token registration)
<code>expo-constants</code>	App constants & manifest values at runtime
<code>expo-crypto</code>	Cryptographic utilities
<code>expo-linking</code>	Deep linking & universal links

Build & Tooling

Package	Purpose
<code>@expo/metro-runtime</code>	Metro bundler runtime for web target
<code>react-native-web</code>	Renders React Native components in the browser
<code>react-dom</code>	React DOM for web target
<code>@expo/ngrok</code>	Local tunnel for testing on physical devices over the internet
<code>eas.json</code>	Expo Application Services build configuration (dev / preview / production)

3 Project Structure

```
balneg/ └─ app/ # File-based routes (expo-router) | └─ _layout.tsx # Root layout -
auth guard, providers | └─ index.tsx # Entry point - splash + role-based redirect |
└─ (auth)/ # Unauthenticated routes (grouped) | | └─ _layout.tsx | | └─ login.tsx #
Login screen | | └─ register.tsx # Registration screen | | └─ onboarding/ #
Onboarding flow for new users | └─ client/ # Client-facing screens (tab navigation) |
| └─ _layout.tsx # Tab bar layout | | └─ dashboard.tsx # Home - stats, AI tips,
activity | | └─ equipment.tsx # Browse & reserve equipment | | └─ reservations.tsx #
My bookings | | └─ workouts.tsx # Workout plans & session logging | | └─ workout-
form.tsx # Create/edit workout form | | └─ notifications.tsx # In-app notifications |
| └─ profile.tsx # Profile & body measurements | └─ admin/ # Admin/Trainer screens |
└─ _layout.tsx | └─ dashboard.tsx # Analytics & overview | └─ equipment-manage.tsx#
CRUD for gym equipment | └─ reservations-manage.tsx # Manage all reservations | └─
users.tsx # View all registered users | └─ components/ # Reusable UI components | └─
NotificationListener.tsx# Background push notification handler | └─ equipment/ #
Equipment-specific components | └─ navigation/ # Custom navigation components | └─
ui/ # Design system (see Section 12) | └─ lib/ # Core utilities & services | └─
auth.tsx # AuthContext + useAuth hook | └─ supabase.ts # Supabase client singleton |
└─ types.ts # TypeScript types (maps to DB schema) | └─ utils.ts # General utility
functions | └─ notifications.ts # Push notification helpers | └─ queue-fallback.ts #
Offline queue / retry logic | └─ constants/ # App-wide constants | └─ colors.ts #
Color palette & design tokens | └─ supabase/ # Backend definitions | └─ functions/ai-
recommendation/ | | └─ index.ts # Edge Function: AI workout recs | └─ migrations/ #
SQL migration files | └─ app.json # Expo app manifest | └─ eas.json # EAS Build
configuration | └─ package.json | └─ .env # Local secrets (not committed to git) | └─
.env.example # Template for .env
```

File-Based Routing (expo-router)

Routes are defined by the file system under `app/`. Special naming conventions apply:

Convention	Meaning
<code>_layout.tsx</code>	Shared layout wrapper for all child routes (Stack, Tabs, Drawer)
<code>(auth)/</code>	Route group — the folder name is invisible in the URL
<code>index.tsx</code>	Default screen when navigating to a folder
<code>[id].tsx</code>	Dynamic route segment

Auth & Role-Based Access

`lib/auth.tsx` exports an `AuthProvider` and `useAuth()` hook. On app mount it:

1. Reads the persisted Supabase session from `SecureStore` (or `localStorage` on web)
2. Fetches the user's `profile` row from the database to determine their **role**
3. Listens for real-time auth state changes (sign in, sign out, token refresh)
4. Exposes `signIn`, `signUp`, `signOut`, `refreshProfile`, and `updateProfile` methods

The root `app/index.tsx` uses the resolved role to redirect the user:

```
// Role-based redirect logic in app/index.tsx
if (!session) → /(auth)/login // Not logged in
if (role === "client") → /client/dashboard // Regular gym member
if (role === "admin"
  || role === "trainer") → /admin/dashboard // Staff
```

Token Storage Strategy

Platform	Storage Method	Security
iOS	expo-secure-store → iOS Keychain	Hardware-encrypted
Android	expo-secure-store → Android Keystore	Hardware-encrypted
Web	localStorage	Browser-scoped

Supabase Realtime

The client is configured with `eventsPerSecond: 10` for live subscriptions. This powers real-time updates such as reservation status changes and incoming notifications without polling.

Database Schema (Supabase / PostgreSQL)

The following tables are defined in `lib/types.ts` and mirror the Supabase PostgreSQL schema:

Table	Description	Key Fields
<code>profiles</code>	Extended user info beyond auth	<code>role</code> , <code>height_cm</code> , <code>weight_kg</code> , <code>fitness_goal</code>
<code>equipment</code>	Gym equipment inventory	<code>name</code> , <code>type</code> , <code>status</code> , <code>brand</code> , <code>muscle_group</code>
<code>exercises</code>	Exercise library	<code>name</code> , <code>muscle_group</code> , <code>tutorial_url</code> , <code>equipment_id</code>
<code>reservations</code>	Equipment booking records	<code>client_id</code> , <code>equipment_id</code> , <code>date</code> , <code>start_time</code> , <code>status</code>
<code>maintenance_log</code>	Equipment maintenance history	<code>equipment_id</code> , <code>staff_id</code> , <code>description</code> , <code>status</code>
<code>workout_plans</code>	AI or manually created plans	<code>goal</code> , <code>difficulty_level</code> , <code>fitt_*</code> fields, <code>is_ai_generated</code>
<code>workout_details</code>	Exercises within a plan	<code>plan_id</code> , <code>exercise_id</code> , <code>sets</code> , <code>reps</code> , <code>rest_seconds</code> , <code>ordering</code>
<code>completed_sessions</code>	Logged workout sessions	<code>plan_id</code> , <code>duration_minutes</code> , <code>performance_metrics</code> , <code>notes</code>
<code>ai_recommendations</code>	AI-generated suggestions	<code>recommended_type</code> (plan/exercise), <code>payload</code> , <code>date_generated</code>
<code>notifications</code>	In-app notification records	<code>title</code> , <code>message</code> , <code>is_read</code> , <code>action_url</code>
<code>measurement_history</code>	Body weight/height tracking	<code>weight_kg</code> , <code>height_cm</code> , <code>measured_at</code>

Enumerations

Enum	Values
<code>UserRole</code>	<code>admin</code> <code>trainer</code> <code>client</code>
<code>EquipmentType</code>	<code>cardio</code> <code>strength</code>
<code>EquipmentStatus</code>	<code>available</code> <code>maintenance</code> <code>occupied</code>

Enum	Values
ReservationStatus	confirmed cancelled completed
MaintenanceStatus	pending fixed
DifficultyLevel	beginner advanced
RecommendationType	plan exercise

6

Environment Variables

Copy `.env.example` to `.env` and fill in your values before running the app:

```
# Required - Supabase Dashboard → Project Settings → API
EXPO_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6...

# Optional - for the AI Edge Function (set in Supabase Secrets, not .env)
# AI_API_KEY=sk-your-openai-api-key
# AI_API_URL=https://api.openai.com/v1/chat/completions
# AI_MODEL=gpt-4o-mini
```



Security Warning

Variables prefixed with `EXPO_PUBLIC_` are embedded at build time and visible to the client.

Never put secret or service-role keys here.

Only the public anon key should go in `.env`.



Where to find your keys

Go to

app.supabase.com

→ Select your project →

Project Settings → API

. Copy the *Project URL* and the *anon / public* key.

7

Prerequisites

Make sure the following tools are installed on your machine before running the app:

Tool	Minimum Version	Check Command	Download
Node.js	≥ 18.x	<code>node -v</code>	nodejs.org
npm	≥ 9.x	<code>npm -v</code>	bundled with Node
Git	Any	<code>git --version</code>	git-scm.com

Tool	Minimum Version	Check Command	Download
Expo CLI	Latest	<code>npx expo --version</code>	via npx (auto)

Optional — Platform Targets

Target	Required Tool
Physical device (Android/iOS)	Expo Go app — install from App Store / Play Store
Android Emulator	Android Studio with an AVD (Android Virtual Device)
iOS Simulator	Xcode (macOS only) with iOS Simulator
Web browser	Any modern browser (Chrome, Edge, Firefox)

1

Clone the Repository

Get the project on your local machine using Git.

```
git clone <your-repository-url>
cd balneg
```

2

Install Dependencies

Install all npm packages listed in `package.json`.

```
npm install
```

3

Set Up Environment Variables

Create your local `.env` file from the provided example template.

```
# Windows (PowerShell)
Copy-Item .env.example .env

# macOS / Linux
cp .env.example .env
```

Then open `.env` and replace the placeholder values with your real Supabase URL and Anon Key.

4

Set Up the Supabase Database

Apply the database schema to your Supabase project.

1. Go to **app.supabase.com** → your project → **SQL Editor**
2. Run the migration files from `supabase/migrations/` in order
3. Verify tables are created under **Table Editor**

5

Start the Development Server

Launch the Expo Metro bundler. A QR code and interactive menu will appear.

```
npm start
# or equivalently:
npx expo start
```

6

Open on Your Target Platform

Once the dev server is running, choose how to open the app:

Key	Platform	Alternative Command
w	Web browser (localhost:8081)	npm run web
a	Android Emulator	npm run android
i	iOS Simulator (macOS only)	npm run ios
Scan QR	Physical device via Expo Go	—

7

Register / Log In

The app opens to the Login screen. Register a new account or log in with existing Supabase credentials. The app automatically redirects you to the correct dashboard based on your **role**.



Quick Start Summary

npm install → copy .env → run migrations → npm start → press w for web

Building for Production (EAS)

The project uses **Expo Application Services (EAS Build)** for creating production-ready binaries.

Install EAS CLI & Login

```
npm install -g eas-cli
eas login
```

Build Profiles

Profile	Command	Output	Notes
development	<code>eas build --profile development</code>	Dev client build	For testing with dev tools
preview	<code>eas build --profile preview --platform android</code>	APK file	Supabase keys pre-baked in eas.json
production	<code>eas build --profile production</code>	AAB / IPA	For Play Store / App Store submission



Production Env Variables

For the production profile, set `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY` in the EAS Dashboard under your project's environment variables instead of hardcoding them.

Supabase Edge Functions

ai-recommendation

Path: `supabase/functions/ai-recommendation/index.ts`

This Deno-based Edge Function generates AI-powered workout recommendations. It:

1. Receives a client profile (fitness goal, weight, height, difficulty preference)
2. Calls an external AI API (OpenAI-compatible endpoint configured via secrets)

3. Returns a structured workout plan or exercise suggestion
4. Stores the result in the `ai_recommendations` table

Deployment Steps

```
# Install Supabase CLI
npm install -g supabase

# Login and link your project
supabase login
supabase link --project-ref your-project-ref

# Deploy the function
supabase functions deploy ai-recommendation

# Set AI environment secrets
supabase secrets set AI_API_KEY=sk-your-openai-key
supabase secrets set AI_API_URL=https://api.openai.com/v1/chat/completions
supabase secrets set AI_MODEL=gpt-4o-mini
```

```
App Launch | ▼ index.tsx (Animated splash – reads auth session & role) | |— Not
logged in —————> /(auth)/login | | |— register.tsx (sign up) | |— onboarding/
(new user flow) | |— role: "client" —————> /client/dashboard | |— equipment.tsx
(browse & book) | |— reservations.tsx (my bookings) | |— workouts.tsx (plans & log)
| |— notifications.tsx (alerts) | |— profile.tsx (body metrics) | |— role: "admin"
/ "trainer" ▶ /admin/dashboard |— equipment-manage.tsx (CRUD) |— reservations-
manage.tsx |— users.tsx (all members)
```

All reusable UI components live in `components/ui/` and form the app's design system:

Button.tsx

Primary, secondary, ghost, and danger variants. Supports loading states and full-width mode.

Input.tsx

Styled text input with label, error message, icon prefix/suffix, and secure-text mode.

Card.tsx

Glass-morphism card container with optional border, shadow, and padding variants.

Badge.tsx

Small status chip — e.g., "Available", "Maintenance", "Confirmed". Color-coded by status.

Toast.tsx

Global toast notification system. Supports success, error, and info types with auto-dismiss.

GlassAlert.tsx

Full-width dismissible alert banner with icon and description. Used for inline warnings.

ActivityGauge.tsx

Circular SVG gauge for displaying workout activity metrics (e.g., completion percentage).

AnimatedBackground.tsx

Full-screen animated gradient background that wraps the entire app via the root layout.

Icon.tsx

NotificationListener.tsx

Wrapper for `lucide-react-native` icons with size, color, and stroke-width control.

Background component that listens for incoming push notifications and foreground alerts.



Generated by Antigravity AI — August 2026

This documentation was auto-generated from source code analysis of the `balneg` workspace. For the latest changes, re-generate from the live codebase.